

Les 8 méthodes de classification du projet B430

Comprendre chaque modèle testé pour prédire la qualité d'un lot — BON ou MAUVAIS — à partir de la **Durée**, de la **Température** et de la **Pression** de broyage.

28 lots

20 BON / 8 MAUVAIS

3 variables

Évaluation : LOOCV

scikit-learn

Ce document explique, méthode par méthode, ce que fait chaque algorithme du dictionnaire `models`, à quoi servent les hyperparamètres choisis dans le code, et pourquoi certains modèles réussissent mieux que d'autres sur ce petit jeu industriel. Les performances reprises sont celles mesurées par validation croisée *Leave-One-Out* (LOOCV) dans le notebook.

MEILLEUR MODÈLE

85,7 %

Random Forest (LOOCV)

VARIABLE LA PLUS
DISCRIMINANTE

Temp.

51 % de l'importance (Gini)

DÉSÉQUILIBRE DES CLASSES

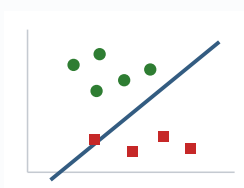
2,5 : 1

corrigé par `balanced`

Deux idées à garder en tête avant de lire. (1) La normalisation. Les trois variables ont des échelles très différentes (minutes, °C, bar). Toutes les méthodes à base de distance — c'est-à-dire tous les SVM — sont précédées d'un `StandardScaler` qui ramène chaque variable à moyenne 0 et écart-type 1. Sans cela, la variable aux plus grands chiffres écraserait les autres. **(2) L'évaluation.** Avec seulement 28 lots, on n'isole pas un jeu de test : on utilise la LOOCV, qui entraîne le modèle 28 fois en laissant à chaque fois *un seul* lot de côté pour le tester. C'est l'usage le plus honnête des données quand elles sont rares — mais chaque erreur pèse alors $1/28 \approx 3,6$ points, donc les écarts de quelques pourcents entre modèles proches ne sont pas forcément significatifs.

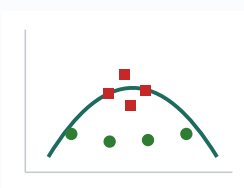
Trois familles de frontières de décision

Un classifieur trace une *frontière* qui sépare l'espace des lots BON de celui des lots MAUVAIS. La forme autorisée pour cette frontière est ce qui distingue le plus les méthodes ci-dessous.



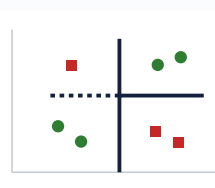
Linéaire

Une droite (un plan en 3D). Simple, robuste, mais incapable d'épouser une séparation courbe.



RBF (gaussien)

Une frontière courbe et lisse qui peut entourer un groupe. Souple, idéal quand les classes ne sont pas alignées.



Arbres / Forêt

Des découpes en escalier, seuil après seuil sur chaque variable. Pas de distance, pas de normalisation requise.

1 SVM

Linéaire

```
SVC(kernel='linear',  
C=1.0)
```

60,7 %

PRINCIPE

La machine à vecteurs de support (SVM) cherche l'hyperplan — ici une droite séparatrice projetée dans l'espace à 3 variables — qui sépare les lots BON des lots MAUVAIS en laissant la plus large « marge » possible de part et d'autre. Intuitivement : elle ne se contente pas de séparer, elle s'écarte le plus possible des deux groupes pour être robuste. Seuls les points les plus proches de la frontière (les *vecteurs de support*) déterminent sa position.

HYPERPARAMÈTRES DE CE CODE

- `kernel='linear'` : la frontière est strictement une droite/plan. Aucune courbure possible.
- `C=1.0` : règle le compromis entre une marge large et le nombre d'erreurs tolérées. Petit C = marge large, on accepte des erreurs (modèle plus général) ; grand C = on force à bien classer chaque point (risque de surapprentissage). 1.0 est une valeur moyenne neutre.
- `random_state=42` : fixe l'aléa interne pour que le résultat soit reproductible d'une exécution à l'autre. N'affecte pas la qualité du modèle, seulement sa répétabilité.

EFFET DE « BALANCED »

Absent ici : les 20 lots BON pèsent mécaniquement plus que les 8 MAUVAIS, donc le modèle est biaisé en faveur de la classe BON.

QUAND ÇA MARCHE / QUAND ÇA COINCE

Excellent quand les deux qualités sont séparables par une simple coupure linéaire. Ici, à 60,7 %, c'est le signe que la frontière réelle n'est **pas** une droite : une variable seule ne suffit pas à trancher proprement.

PERFORMANCE LOOCV

60,7 % — le plus faible des SVM utilisables : confirme qu'un modèle purement linéaire est trop rigide pour ce problème.

2 SVM Linéaire (bal.)

```
kernel='linear',  
class_weight='balanced'
```

67,9 %

PRINCIPE

Exactement le même hyperplan à marge maximale que le n°1, mais on rééquilibre l'importance des classes pendant l'entraînement. Le modèle est davantage pénalisé lorsqu'il se trompe sur un lot MAUVAIS, classe minoritaire.

HYPERPARAMÈTRES DE CE CODE

- `class_weight='balanced'` : attribue à chaque classe un poids inversement proportionnel à son effectif. Concrètement, une erreur sur un MAUVAIS « coûte » ici environ 2,5 fois plus cher qu'une erreur sur un BON (20/8). La frontière se décale pour mieux protéger la classe rare.
- `C=1.0`, `kernel='linear'`, `random_state=42` : identiques au n°1.

POURQUOI « BALANCED » ICI

Le jeu est déséquilibré (20/8). Sans correction, le modèle peut maximiser sa précision globale en disant « BON » presque partout. Le rééquilibrage l'oblige à prendre la détection des MAUVAIS au sérieux — précisément l'enjeu qualité.

QUAND ÇA MARCHE / QUAND ÇA COINCE

Le gain de +7 points sur le n°1 montre que le déséquilibre nuisait réellement. Mais la frontière reste une droite : le plafond de performance d'un modèle linéaire n'est pas franchi.

PERFORMANCE LOOCV

67,9 % — meilleur que le linéaire brut grâce à l'équilibrage, mais encore limité par la rigidité de la droite.

3 SVM RBF

```
kernel='rbf', C=1.0,  
gamma='scale'
```

71,4 %

PRINCIPE

Le noyau RBF (*Radial Basis Function*, gaussien) permet à la SVM de tracer une frontière **courbe**. L'astuce mathématique du « kernel trick » revient à projeter les lots dans un espace de très grande dimension où ils deviennent séparables par un plan, sans jamais calculer cette projection explicitement. Résultat visible : la frontière peut entourer un groupe de points au lieu de le couper en ligne droite.

HYPERPARAMÈTRES DE CE CODE

- `kernel='rbf'` : autorise les frontières courbes et lisses.
- `gamma='scale'` : contrôle la « portée » de chaque point. Grand gamma = influence locale, frontière très ondulée (risque de surapprentissage) ; petit gamma = influence large, frontière presque linéaire.
`'scale'` calcule automatiquement gamma à partir de la variance des données — un choix par défaut raisonnable.
- `C=1.0` : même compromis marge/erreurs que précédemment.

EFFET DE « BALANCED »

Absent ici : la souplesse de la frontière aide, mais le modèle reste tiré vers la classe majoritaire BON.

QUAND ÇA MARCHE / QUAND ÇA COINCHE

Le RBF est le bon réflexe quand on ignore la forme de la frontière. Sur 28 points seulement, il faut surveiller le surapprentissage, mais ici il bat déjà le linéaire de 11 points : la séparation est bien non linéaire.

PERFORMANCE LOOCV

71,4 % — meilleur que tous les linéaires : la flexibilité du noyau gaussien paie.

4 SVM RBF (bal.)

```
kernel='rbf', gamma='scale',  
class_weight='balanced'
```

75,0 %

PRINCIPE

La frontière courbe du RBF (n°3) combinée au rééquilibrage des classes. On garde la souplesse gaussienne tout en forçant le modèle à ne pas négliger les lots MAUVAIS.

HYPERPARAMÈTRES DE CE CODE

- `class_weight='balanced'` : surpondère la classe MAUVAIS ($\approx \times 2,5$), comme au n°2.
- `gamma='scale', C=1.0, kernel='rbf'` : identiques au n°3.

POURQUOI « BALANCED » ICI

Même logique que le n°2 : sur un jeu 20/8, protéger la classe rare est l'objectif métier. Combiné à une frontière souple, c'est généralement le meilleur compromis SVM « par défaut ».

QUAND ÇA MARCHE / QUAND ÇA COINCHE

+3,6 points sur le RBF non équilibré : le gain est réel mais modeste, signe que sur ce jeu le déséquilibre pèse moins une fois la frontière déjà courbe. C'est le meilleur SVM « non optimisé » du lot.

PERFORMANCE LOOCV

75,0 % — solide ; une recherche d'hyperparamètres (GridSearch) le pousse même à 82,1 % dans le notebook.

5 SVM Poly deg=2 (bal.)

```
kernel='poly', degree=2,  
class_weight='balanced'
```

42,9 %

PRINCIPE

Le noyau polynomial autorise une frontière en forme de courbe polynomiale. Avec `degree=2`, la frontière peut être une parabole ou une ellipse — utile si les lots BON forment, par exemple, une « cuvette » de bons réglages entourée de mauvais.

HYPERPARAMÈTRES DE CE CODE

- `degree=2` : degré du polynôme. Plus le degré monte, plus la frontière peut se tordre — et plus le risque de surapprentissage augmente.
- `class_weight='balanced'` : surpondère la classe MAUVAIS.
- Note : `C` et `gamma` restent à leurs valeurs par défaut, non réglés ici ; le noyau polynomial y est très sensible, ce qui explique en partie le mauvais résultat.

QUAND ÇA MARCHE / QUAND ÇA COINCE

Le polynomial est capricieux : il faut souvent régler finement `degree`, `C`, `gamma` et `coef0` ensemble. Laissé en réglages par défaut sur 28 points, il colle mal à la structure réelle des données.

PERFORMANCE LOOCV

42,9 % — sous la barre symbolique : pour ce jeu, un noyau polynomial non réglé est contre-productif.

6 SVM Poly deg=3 (bal.)

```
kernel='poly', degree=3,  
class_weight='balanced'
```

35,7 %

PRINCIPE

Identique au n°5 mais avec un polynôme de degré 3 : la frontière peut se tordre encore davantage (formes en « S », doubles courbures). Plus de liberté géométrique, donc plus de risque d'apprendre le bruit plutôt que le signal.

HYPERPARAMÈTRES DE CE CODE

- `degree=3` : un degré de plus que le n°5. Sur peu de données, cette flexibilité supplémentaire dégrade généralement la généralisation.
- `class_weight='balanced'` : surpondère la classe MAUVAIS.

QUAND ÇA MARCHE / QUAND ÇA COINCE

Cas typique de surapprentissage : un modèle plus flexible (degré 3) fait *pire* qu'un moins flexible (degré 2). Avec 28 lots, augmenter la complexité sans plus de données ne fait qu'amplifier le bruit.

PERFORMANCE LOOCV

35,7 % — le plus mauvais score du tableau ; sert d'illustration claire qu'« plus complexe » n'est pas « meilleur ».

7 NuSVC RBF (bal.)

```
NuSVC(kernel='rbf', nu=0.5,  
balanced)
```

78,6 %

PRINCIPE

NuSVC est une SVM RBF reformulée : au lieu de régler le paramètre abstrait **C**, on règle **nu**, qui a un sens concret. La frontière reste de même nature (courbe gaussienne), mais le contrôle du compromis se fait par une grandeur interprétable.

HYPERPARAMÈTRES DE CE CODE

- **nu=0.5** : borne **supérieure** sur la fraction d'erreurs tolérées et borne **inférieure** sur la fraction de vecteurs de support. À 0,5, le modèle s'autorise jusqu'à ~50 % de points « difficiles » — réglage souple, adapté à des classes qui se chevauchent.
- **kernel='rbf'** : frontière courbe, comme les n°3 et 4.
- **class_weight='balanced'** : surpondère la classe MAUVAIS.

POURQUOI « BALANCED » ICI

Toujours le même motif 20/8 : on protège la classe minoritaire. La paramétrisation par **nu** donne en plus une prise directe sur le nombre de points-frontière, utile quand les groupes se recouvrent.

QUAND ÇA MARCHE / QUAND ÇA COINCE

Meilleur SVM du lot (78,6 %). La formulation **nu** tombe souvent juste sans réglage fin, ce qui est précieux sur un petit jeu. Reste sensible au choix de **nu** : trop élevé, il refuserait de séparer.

PERFORMANCE LOOCV

78,6 % — le meilleur des modèles SVM testés sans optimisation par grille.

8 Random Forest

```
RandomForestClassifier(n_estimators=100)
```

85,7 %

PRINCIPE

Ce n'est pas une SVM. Une forêt aléatoire construit 100 arbres de décision, chacun posant une succession de questions à seuil (« Temp. > 80 ? », « Pression < 3 ? ») pour découper l'espace en rectangles. Chaque arbre vote, et la majorité l'emporte. La diversité des arbres — entraînés sur des sous-échantillons et des sous-ensembles de variables différents — corrige les erreurs individuelles : c'est la « sagesse de la foule ».

HYPERPARAMÈTRES DE CE CODE

- `n_estimators=100` : nombre d'arbres dans la forêt. Plus il y en a, plus le vote est stable (au prix du temps de calcul) ; 100 est un standard robuste.
- `random_state=42` : fige le tirage aléatoire des arbres pour un résultat reproductible.
- Pas de `class_weight` ici : la forêt s'en sort sur ce déséquilibre sans rééquilibrage explicite (on pourrait toutefois l'ajouter).

ATOUT SPÉCIFIQUE

Pas de distance, donc **pas besoin de normaliser**, et elle fournit gratuitement l'*importance des variables* : on apprend ici que la Température explique 51 % de la décision, la Pression 27 %, la Durée 22 %.

QUAND ÇA MARCHE / QUAND ÇA COINCE

Très performante avec peu de variables et des frontières « en escalier », ce qui correspond bien à des seuils de procédé. Limite : moins lisible qu'une simple droite, et elle peut surapprendre si les arbres ne sont pas bornés — mais le vote de 100 arbres limite ce risque.

PERFORMANCE LOOCV

85,7 % — meilleur modèle du projet, devant tous les SVM.

Tableau récapitulatif

#	MODÈLE	FRONTIÈRE	ACCURACY LOOCV	FORCES	FAIBLESSES
8	Random Forest	Escaliers (arbres)	85,7 %	Meilleur score ; pas de normalisation ; importance des variables	Moins interprétable qu'une droite ; risque de surapprentissage si non bornée
7	NuSVC RBF (bal.)	Courbe (gaussienne)	78,6 %	Meilleur SVM ; nu interprétable ; bon sans réglage	Sensible au choix de nu
4	SVM RBF (bal.)	Courbe (gaussienne)	75,0 %	Souple + classe rare protégée ; monte à 82 % après GridSearch	Gamma/C par défaut sous-optimaux
3	SVM RBF	Courbe (gaussienne)	71,4 %	Capte la non-linéarité	Biaisé vers la classe BON majoritaire
2	SVM Linéaire (bal.)	Droite / plan	67,9 %	Simple ; classe rare protégée	Frontière trop rigide
1	SVM Linéaire	Droite / plan	60,7 %	Le plus simple et le plus rapide	Trop rigide ; biaisé vers BON
5	SVM Poly deg=2 (bal.)	Courbe polynomiale	42,9 %	Capable de formes elliptiques	Très sensible aux réglages ; non optimisé ici
6	SVM Poly deg=3 (bal.)	Courbe polynomiale	35,7 %	Frontière très flexible	Surapprentissage net ; pire que le degré 2

Les barres sont proportionnelles à l'accuracy. Toutes les valeurs sont issues de la validation croisée Leave-One-Out sur les 28 lots.

Conclusion

Pourquoi la Random Forest devance les SVM ici. La frontière qui sépare un bon broyage d'un mauvais ressemble davantage à une combinaison de *seuils de procédé* (« température dans telle plage ET pression sous tel niveau ») qu'à une surface géométrique lisse. Or c'est exactement ce que des arbres de décision modélisent naturellement, par découpes successives. La forêt cumule en plus deux avantages décisifs sur un petit jeu : elle n'exige aucune normalisation et son vote sur 100 arbres amortit le bruit que les SVM, eux, doivent absorber dans une frontière unique.

Le rôle central de la Température. La forêt attribue 51 % de l'importance à la **Temp. (°C)**, contre 27 % à la Pression et 22 % à la Durée. Autrement dit, la moitié de la décision se joue sur la température : c'est la variable à surveiller en priorité sur la ligne. Cela explique aussi pourquoi les modèles linéaires plafonnent — une seule droite peine à exploiter une variable aussi dominante en présence des deux autres.

Réserve méthodologique. Avec 28 lots, chaque lot mal classé vaut 3,6 points d'accuracy. Les écarts entre modèles voisins (par ex. RBF 71,4 % vs RBF bal. 75,0 %) reposent sur 1 ou 2 lots et ne sont donc pas statistiquement robustes. La hiérarchie générale — forêt et RBF en tête, polynômes en queue — est fiable ; le classement fin, beaucoup moins. Pour conclure solidement, il faudrait davantage de lots, en particulier de la classe MAUVAIS.